Avatar, data mining with Skype

Dave Cuthbert
Feb 15 · 3 min read

There is a lot of useful information within the files that make up a Skype account. You just need to prospect a bit to find the gems. Skype's storage backend is really just a series of sqlite databases so with a little digging and some SQL it isn't too difficult to mine that data and pull out whatever nugget you're looking for. In this post we'll see how to manipulate the database and use that knowledge to collect pictures of the people in your Skype contact list.

Find the database

Skype's data is kept in a few different databases on disk. They are all stored together in the same directory. On Windows the path will be something like:

```
C:\Users\USERNAME\AppData\Roaming\Skype\SKYPE_NAME
```

These are mostly SQLite databases. Unlike MySQL or PostgreSQL, SQLite stores all of each database within a single file. The database and tables we will need for this exercise are in the file named `main.db`.

Determine the structure of main.db

Let's see what the tables look like inside. Open up the SQLite command line tool, `sqlite3`. Then query the SQLite metadata to get a list of the tables in this database file.

```
skype> sqlite3 main.db
      SQLite version 3.8.7.4 2014-12-09 01:34:36
      Enter ".help" for usage hints.
      sqlite> SELECT name FROM sqlite_master WHERE
type='table';
            DbMeta
            AppSchemaVersion
            Contacts
            <Many more tables not listed here>
```

'Contacts' looks is a likely spot to store information on the avatars. Take a look at the fields by running the SQLite `.schema` dot command. (The command output has been shortened for clarity.)

```
sqlite> .schema contacts
CREATE TABLE Contacts (id INTEGER NOT NULL PRIMARY KEY,
is_permanent INTEGER, type INTEGER, skypename TEXT,
pstnnumber TEXT, aliases TEXT, fullname TEXT, birthday
INTEGER, gender INTEGER, languages TEXT, country TEXT,
   <many more columns>
, avatar_url TEXT, avatar_url_new TEXT, avatar_hiresurl
TEXT, avatar_hiresurl_new TEXT,
   <many more columns>);
CREATE INDEX IX_Contacts_skypename ON Contacts (skypename);
CREATE INDEX IX_Contacts_buddystatus ON Contacts
(buddystatus);
CREATE INDEX IX_Contacts_type ON Contacts (type);
CREATE INDEX IX_Contacts_displayname_skypename ON Contacts
(displayname COLLATE NOCASE, skypename);
```

Get the URL list

Tucked away towards the end of the column list is one which sounds promising: `avatar_url`. The values for that field can be inspected by running a simple SQL query:

```
SELECT avatar_url FROM contacts WHERE avatar_url != 'None';
```

Depending on your contact list there could be a lot of urls. To output the list to a file called 'urllist' first run a little data output setup step in SQLite and then rerun the query.

```
sqlite> .mode csv
sqlite> .output urllist
sqlite> SELECT avatar_url FROM contacts WHERE avatar_url !=
'None';
sqlite>
```

Fetch the avatar files

Now that you have the list of urls, the quickest way to fetch the files is to pass the output file into wget like this:

```
wget -i urllist
```

NOTE: Depending on how many avatar files there are, it is probably a good idea to first create a subdirectory to hold them all before running wget.

Clean up the file names

When `wget` retrieves an avatar file it gives it a name that is based on the URL used to fetch the picture. The names take two forms and look something like this list of examples:

```
a_skype_name@auth_key=22222222
another_skype_name@auth_key=12345678
public
public.1
really_yet_another_skype_name@auth_key=234567890
```

These names aren't ideal to work with, but it is easy to to clean them up a bit with a couple of lines of `bash` script.

First add the 'jpg' extension to the public* files:

```
for f in $(ls pub*) ; do mv ${f} ${f}.jpg ; done
```

Then, chop off the “@auth_key=” from the other files and give them the ‘jpg’ extension too.

```
for f in $(ls *@*) ; do mv ${f} $(ls ${f}|cut -d'@' -f1).jpg
; done
```

All that’s left is to take a look at the pictures and delete the ones which you don’t like. Of course, it’s never to soon to see if there are more useful bits of data in the other database files …

Happy hunting!